

CLASE - 05/11

AGILIDAD



(esta clase no tuve la notebook, encima después me tuve que retirar. Así que se jugó alejo las anotaciones)

PROBLEMAS DE LOS MÉTODOS CLÁSICOS

PROBLEMAS DE LOS MÉTODOS CLÁSICOS

- Burocracia
- Impredecibilidad de los requerimientos
- Métodos predictivos, rígidos

FRAMEWORKS ÁGILES

FRAMEWORKS ÁGILES

- Son adaptativos, no predictivos
- Orientados a las personas, no a los procesos
- Importancia del factor humano para el éxito del proyecto

Metodologías tradicionales	Metodologías ágiles
Predictivos	Adaptativos
Orientados a procesos	Orientados a personas
Proceso rígido	Proceso flexible
Se concibe como un proyecto	Un proyecto es subdividido en varios proyectos más pequeños
Poca comunicación con el cliente	Comunicación constante con el cliente
Entrega de software al finalizar el desarrollo	Entregas constantes de software
Documentación extensa	Poca documentación

FRAMEWORKS AGILES

- Aparecen al final del siglo XX como una reacción a los métodos clásicos
- En febrero del 2001 se reúnen los principales representantes y firman el "Manifiesto Ágil"

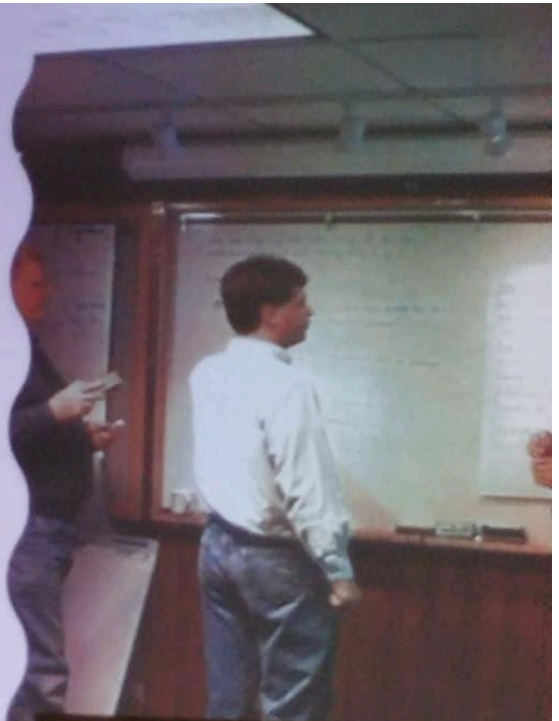


COMPARATIVA TRADICIONAL VS ÁGILES

Metodologías tradicionales	Metodologías ágiles
Predictivos	Adaptativos
Orientados a procesos	Orientados a personas
Proceso rígido	Proceso flexible
Se concibe como un proyecto	Un proyecto es subdividido en varios proyectos más pequeños
Poca comunicación con el cliente	Comunicación constante con el cliente
Entrega de software al finalizar el desarrollo	Entregas constantes de software
Documentación extensa	Poca documentación

MANIFIESTO ÁGIL

VALORES(4)



MANIFIESTO ÁGIL - VALORES

- **Individuos e interacciones**, por sobre procesos y herramientas
- **Software funcionando**, por sobre documentación detallada
- **Colaboración con el cliente**, por sobre negociación contractual
- **Responder al cambio**, por sobre seguir un plan

<https://agilemanifesto.org/>

<https://agilemanifesto.org/>

PRINCIPIOS(12)

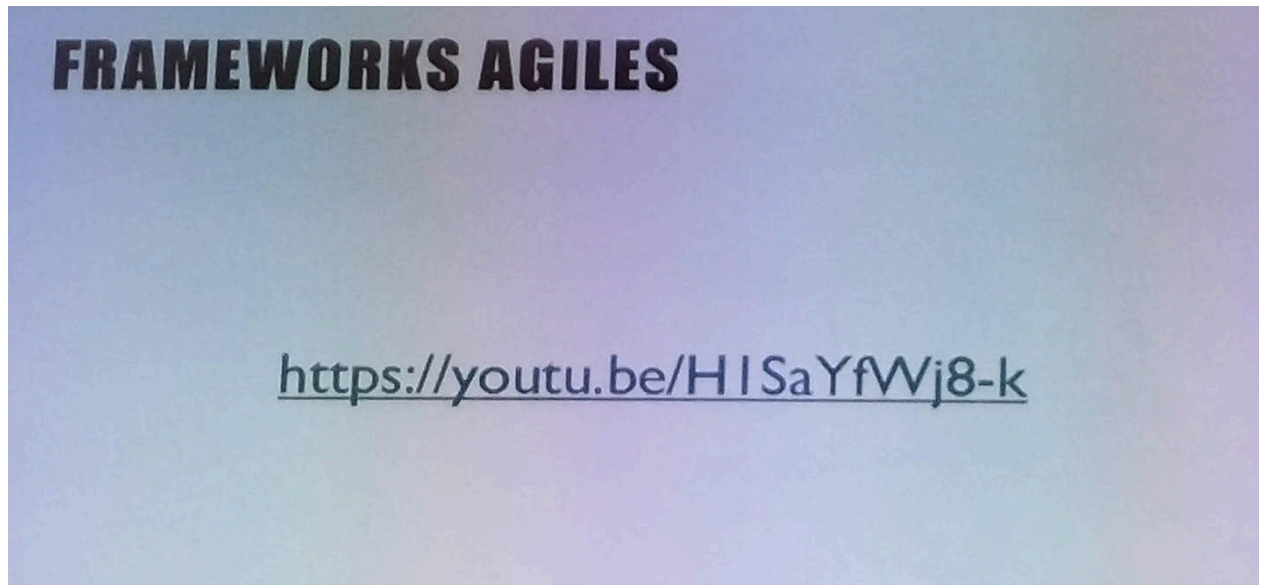
MANIFIESTO ÁGIL - PRINCIPIOS

- Nuestra prioridad más alta es **satisfacer al cliente** a través de la **entrega temprana y continua** de software valioso.
- El **cambio** de requerimientos **es bienvenido**, aún tarde en el desarrollo. Los procesos ágiles aprovechan el cambio para una **ventaja competitiva** del cliente.
- **Entregar software funcionando frecuentemente**, desde un par de semanas a un par de meses, con la mayor brevedad preferentemente.
- Las personas del **negocio y los desarrolladores trabajan diariamente** juntas en el proyecto.
- Construir proyectos alrededor de **individuos motivados**. Brindarles el entorno y el soporte que necesitan, y confiar en ellos para obtener el trabajo realizado.
- El método más eficiente y efectivo para transmitir información hacia y dentro del equipo de desarrollo, es la **conversación cara a cara**.

MANIFIESTO ÁGIL - PRINCIPIOS

- Software funcionando es la primera **medida de progreso**.
- Los procesos ágiles promueven el **desarrollo sostenible**. Los auspiciantes, desarrolladores, y usuarios deben ser capaces de mantener un ritmo constante indefinidamente.
- La atención continua a **la excelencia técnica y el buen diseño** aumenta la agilidad.
- La **simplicidad** – el arte de maximizar el trabajo no realizado – es esencial.
- Las mejores arquitecturas, requerimientos, y diseños emergen de **equipos auto organizados**.
- A intervalos regulares, **reflexiona como ser más efectivo**, y luego ajusta y afina su comportamiento en concordancia.

FRAMEWORKS ÁGILES



Nos hizo ver este video 45 minutos y hablamos sobre el mismo después.

[¿Qué Metodologías Ágiles existen? Vamos con las que fundaron el Manifiesto: FDD, DSDM, XP, etc.](https://youtu.be/HISaYfVWj8-k)

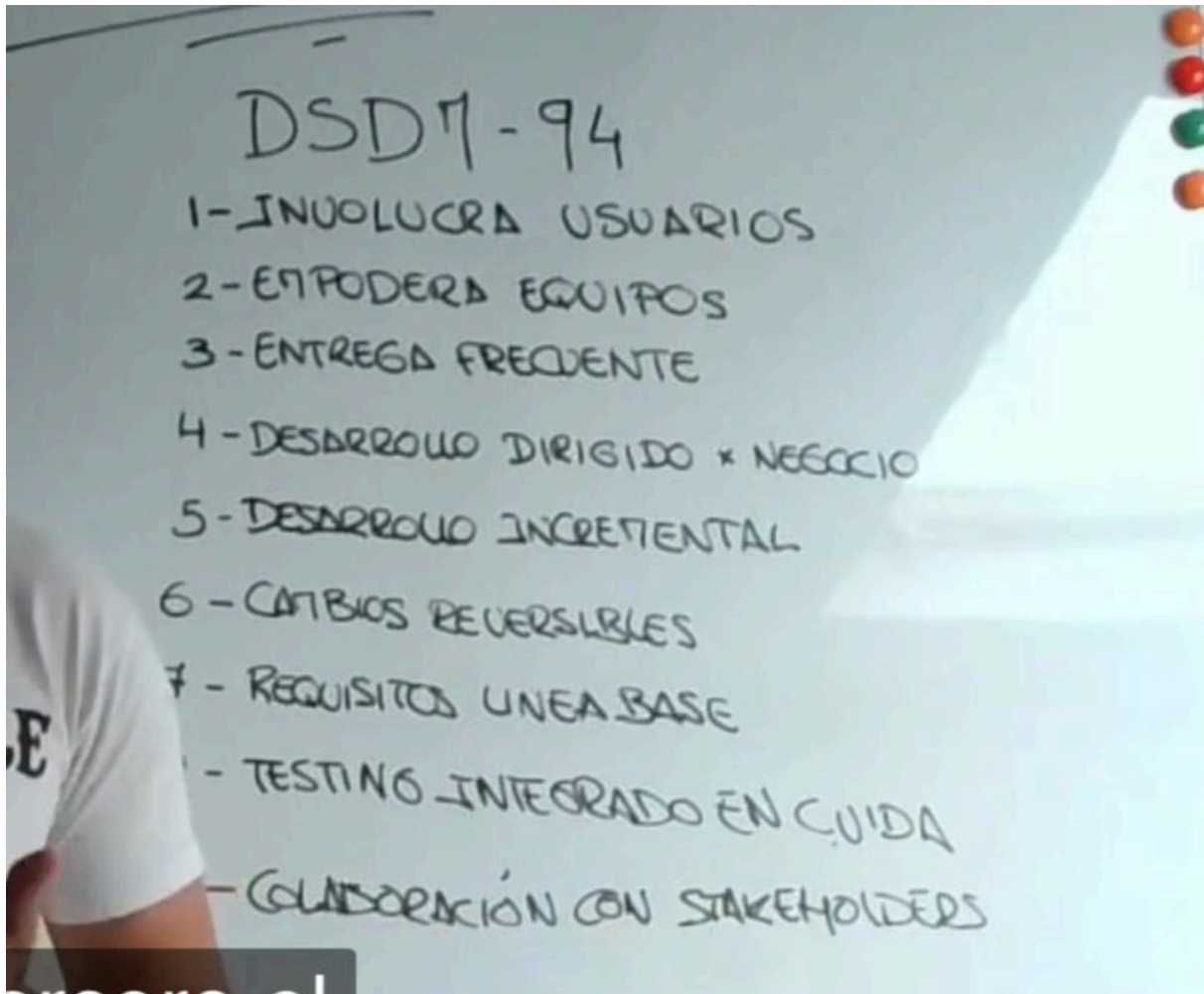
Fueron saliendo oficialmente desde 1994 a 1999 en orden

- 1- DSDM - Dynamic Systems Development Method
- 2- Scrum
- 3- XP - Extreme Programming
- 4- FDD - Feature Driven-Development
- 5- Crystal Methods
- 6- ASD - Adaptive Software Development

Video Resumen (Jero y gonza)

a mano

1- DSDM - Dynamic Systems Development Method



Se usaba Moscow
Concepto de timebox

2- Scrum

Nace en la OPSLA

Se presenta antes en un contexto que no era el software

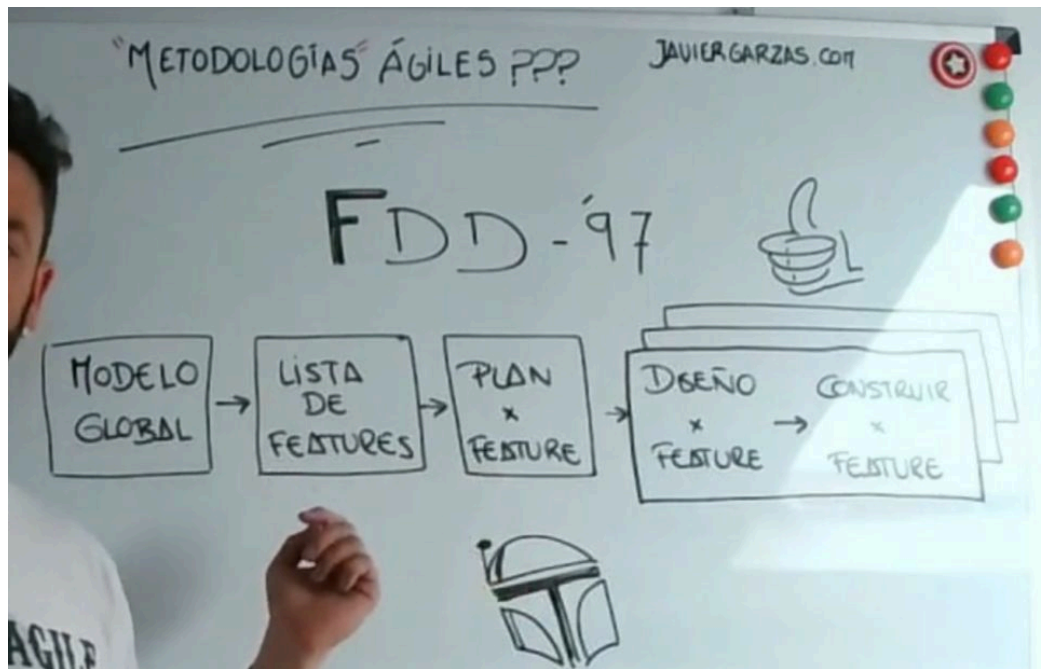
Popular y conocido

3 creadores

3- XP

5 personas en el manifesto

4- FDD - Feature Driven-Development



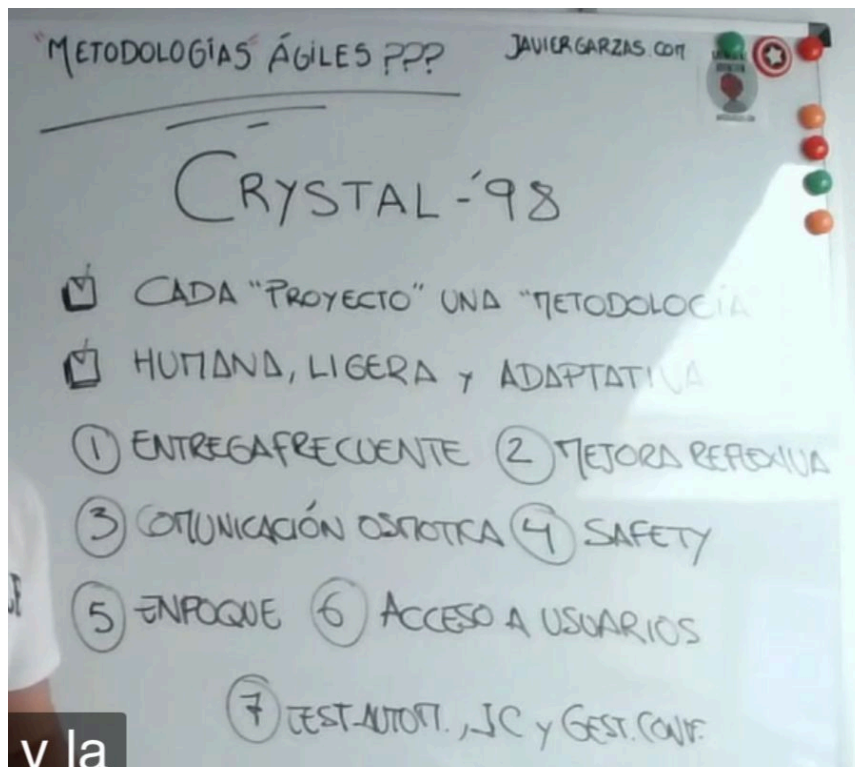
1 solo en el manifesto

Tipo cascada, pero sigue siendo ágil

Se usa cuando no queremos entrar a un iterativo e incremental muy muy ágil

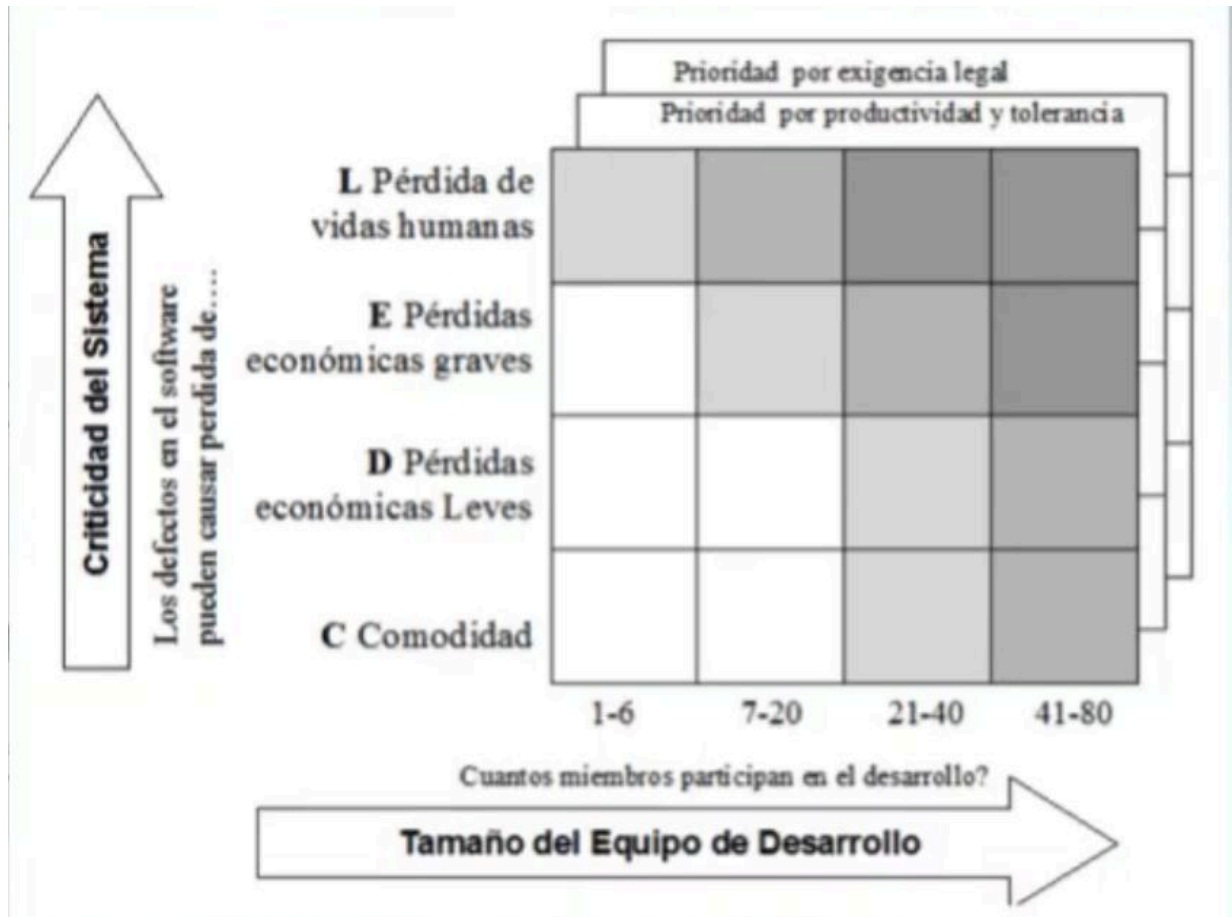
Híbrido de fwk ágil

5- Crystal Methods

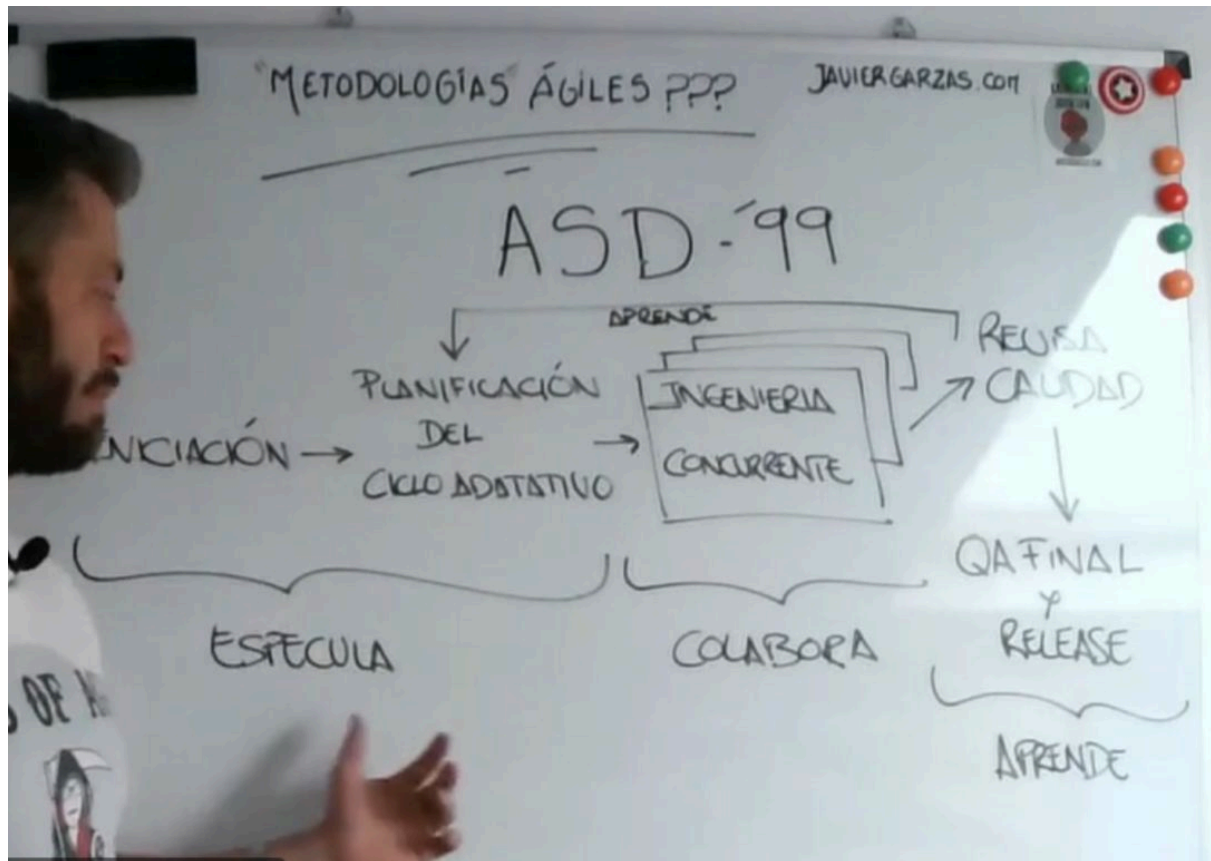


y la

Un firmante del ágil
 Una metodología por proyecto
 Ejes de criticidad y tamaño
 Familia de fmwk
 Hacia analogía con los minerales, por colores y dureza
 Marcado por ser Humanas, ligeras y adaptativas
 Comunicación cara a cara
 Sector seguro



6- ASD - Adaptive Software Development



Una firma en el manifiesto

Enfocado en un ciclo

Evitar de pensar que hay una sola forma de hacer todo, encontrar una forma que se adapte al problema (situación)

Tiene un ciclo de aprendizaje

Iterativo e incremental

Video resumen (Alejo + gpt)

Resumen del video sobre **metodologías ágiles más allá de Scrum**:

El expositor propone **no usar la palabra “metodología”** porque, según él, tiene **connotaciones rígidas y prescriptivas**, como si existiera una única forma correcta de trabajar y los equipos solo debieran seguir pasos fijos.

En su lugar, **prefiere hablar de “frameworks” o “marcos de trabajo”**, e incluso pone “*metodología*” entre comillas cuando la menciona.

La idea es que un framework sea algo **flexible y adaptable al contexto**, donde **el equipo asume la responsabilidad de ajustar las prácticas** según sus necesidades, en lugar de seguir un método cerrado.

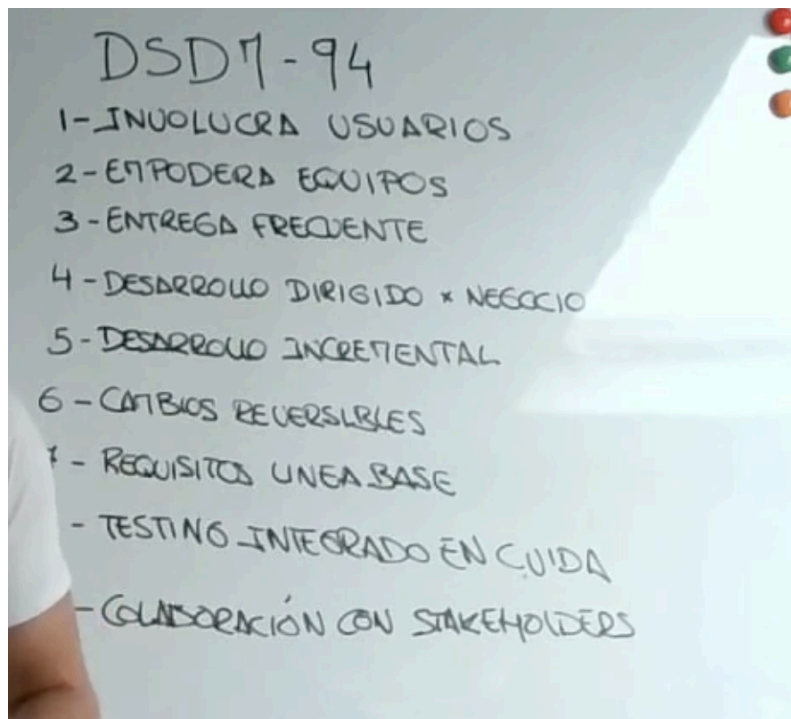
También enfatiza el enfoque de **“chorri picking” (o cherry picking)**:

Tomar lo mejor de cada framework o práctica, combinarlas, y construir tu propia manera de trabajar.

Presenta **seis frameworks “clásicos”** que sirvieron de base para el **Manifiesto Ágil (2001)**, todos desarrollados entre **1994 y 1999**, y que siguen aportando ideas útiles:

1. **DSDM (Dynamic Systems Development Method, 1994)**

- Primer framework considerado “ágil”.
- Enfocado en la participación de usuarios, empoderamiento del equipo, entregas frecuentes y testing continuo.
- Introduce prácticas como *timeboxing* y la priorización *MoSCoW* (Must, Should, Could, Won't)
- Muy orientado al negocio y a la colaboración con *stakeholders*.



2. Scrum (1995)

- Framework más conocido y popularizado.
- Basado en iteraciones cortas, roles definidos (PO, SM, Equipo), y revisión continua.
- **Reuniones ceremoniales:** Daily, Review, Retrospective.
- Tuvo tres firmantes del manifiesto: Ken Schwaber, Jeff Sutherland y Mike Beedle.
- No fue pensada para desarrollo de software, habla de como **gestionar proyectos**. Todas las demás propuestas se enfocan en SW.

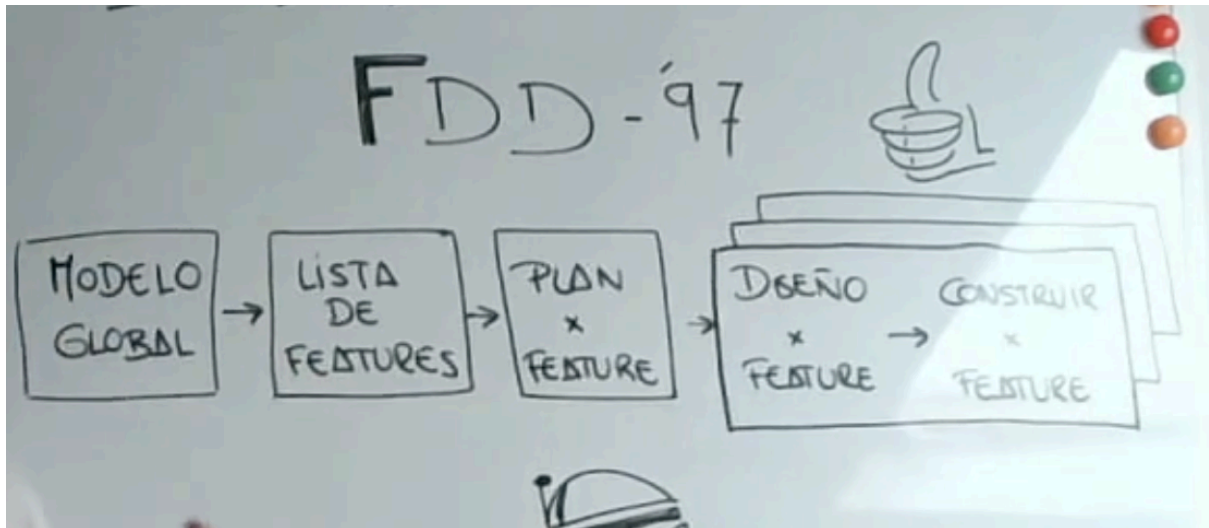
3. Extreme Programming – XP (1996)

- Muy influyente en el desarrollo de software ágil.
- Enfatiza buenas prácticas técnicas: *pair programming*, *TDD*, *refactoring*, *continuous integration*.
- Fue la comunidad más representada en la firma del manifiesto.

4. FDD (Feature-Driven Development, 1997)

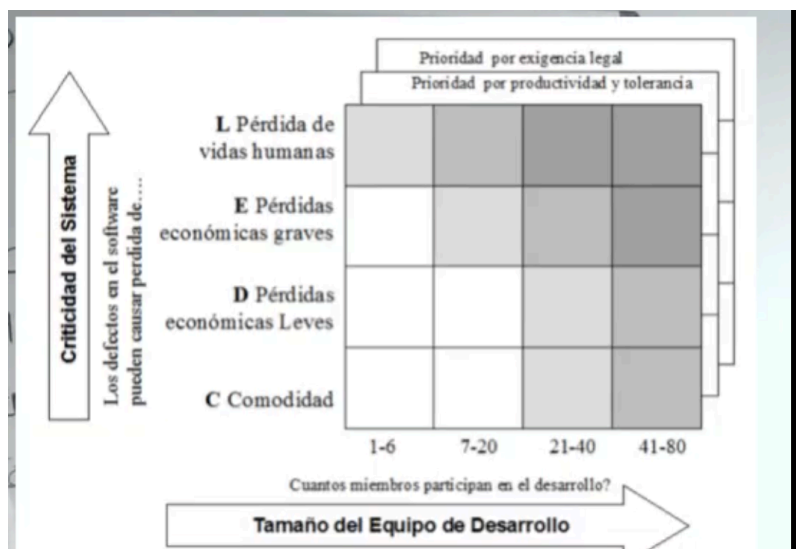
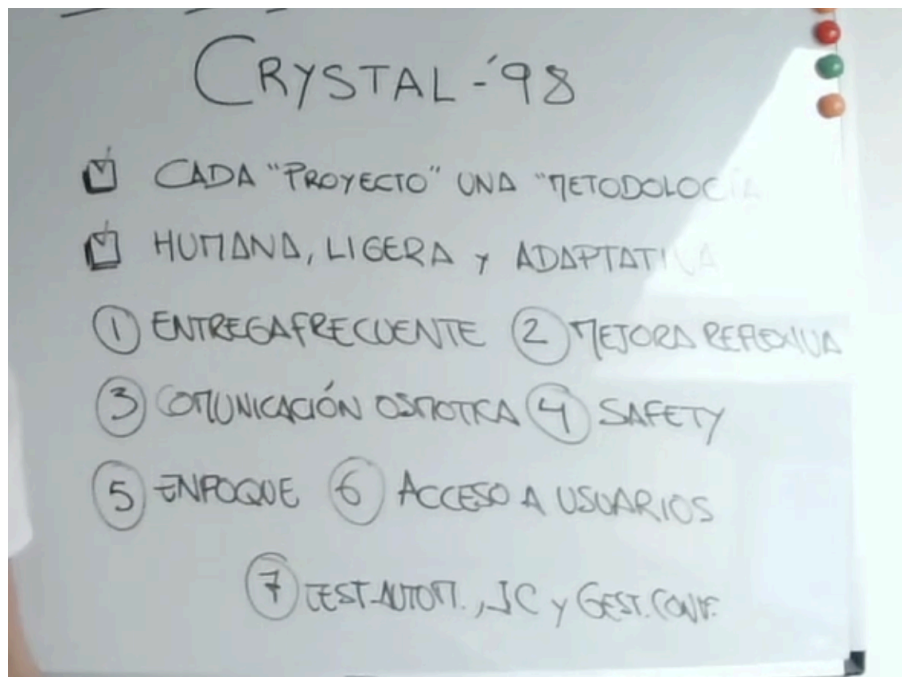
- Enfocado en construir el software por *features* (funcionalidades).
- Tiene un enfoque más secuencial y planificado que otros frameworks, útil cuando no se puede aplicar una agilidad total.

- Su creador oficial es Jeff De Luca.
- **Diferencia:** es el más **estructurado y cercano a un modelo híbrido entre cascada y ágil**, útil cuando se necesita más previsibilidad o control.



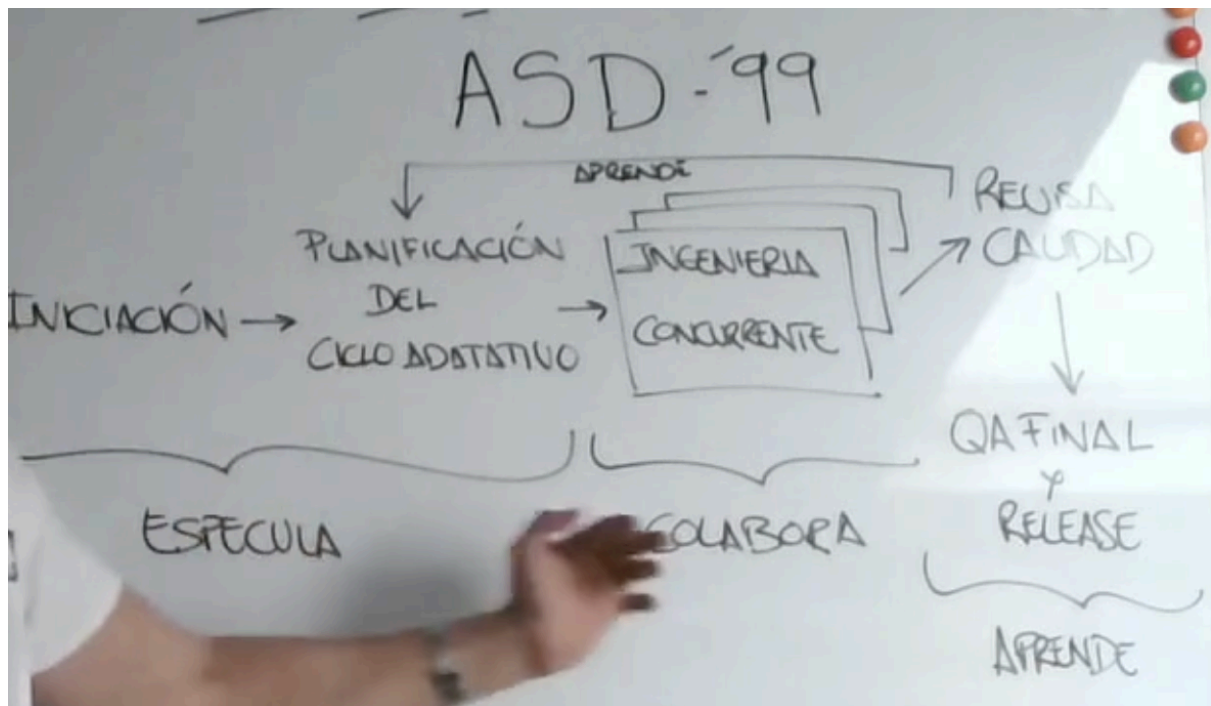
5. **Crystal Methods (1998)** Diferencia: es el más estructurado y cercano a un modelo híbrido entre cascada y ágil, útil cuando se necesita más previsibilidad o control.4

- Propuesto por Alistair Cockburn.
- Familia de métodos / frameworks (Crystal Clear, Yellow, Orange, etc.) adaptados al tamaño y criticidad del proyecto.
- Se centra en las personas, la comunicación, la seguridad psicológica y la entrega frecuente.
 - i. seguridad = poder proponer
- Introduce la “**comunicación osmótica**” y la “**mejora reflexiva**”, antecesora de las retrospectivas.
 - i. **osmótica = cara a cara**



6. ASD (Adaptive Software Development, 1999)

- Creado por Jim Highsmith.
- Ciclo basado en especulación, colaboración y aprendizaje continuo.
- Promueve adaptar el proceso a cada problema, evitando soluciones únicas.
- Tiene similitudes con FDD pero con un enfoque más iterativo e incremental.



Finalmente, el autor resalta que **todos estos frameworks aportaron ideas que dieron origen al Manifiesto Ágil**, y que conocerlos permite aplicar prácticas útiles fuera del marco limitado de Scrum. Invita a experimentar y adaptar cada enfoque según el contexto real del equipo.

En que se parecen todas? -> Todas se alinean con el manifiesto ágil

Gaona nos recomienda que redactemos un párrafo con propuestas principales de cada uno. Podría preguntarle alguna idea de alguno de estos métodos ágiles.

FRAMEWORKS ÁGILES

FRAMEWORKS AGILES

- Scrum
- XP – Extreme Programming
- Crystal
- ASD (Adaptive Software Development)
- FDD (Feature Driven Development)
- DSDM (Dynamic System Development Method)

<https://youtu.be/H1SaYFWj8-k>

XP - PROGRAMACIÓN EXTREMA

XP – PROGRAMACIÓN EXTREMA

- Pensada para dominios de problema muy **dinámicos**
- Orientada a **pequeños equipos** de trabajo
- Requiere también un **equipo de trabajo extendido**
- **Prioriza la prueba**
- Es una “**metodología liviana**”

extreme
Programming
explained
EMBRACE CHANGE

Kent Beck

- Equipos de 2 a 10 personas, no más dado que XP no incluye tareas de coordinación.
- Equipo de trabajo extendido = implicar a personas del negocio.
- Metodología liviana = conjunto de valores, principios y prácticas. No una secuencia de pasos
- Equipos bien autoorganizados. Roles de coaches o “líderes” que motiven a trabajar con este enfoque
- Centrado en las personas

VALORES

XP – VALORES

Simplicidad

Comunicación

Retroalimentación

Coraje

Respeto

- Se resalta la simplicidad -> se debe siempre elegir la solución más simple posible. Tratar de resolver los problemas presentes o lo que me piden y no trabajar sobre problemas inexistentes
- Comunicación -> cara a cara y en un lugar físico
- Retroalimentación -> el software funcionando es lo que me marca si estoy avanzado o no
- Coraje -> transparente, no debería tener miedo de comunicar si algo va mal (inclusive hacia el cliente)
- Respeto -> también se lo conoce como "seguridad psicológica". Nadie del equipo debería tener miedo a expresar su opinión

SCCRR (mnemotecnica)



Los valores de la programación extrema

La Programación Extrema (XP) se basa en valores. Las reglas que acabamos de examinar son la extensión y consecuencia natural de maximizar nuestros valores. XP no es realmente un conjunto de reglas, sino una forma de trabajar en armonía con los valores personales y corporativos. Comience con los valores de XP que se enumeran aquí y luego agregue los suyos reflejándolos en los cambios que realice a las reglas.

Simplicidad: Haremos lo necesario y lo que se solicita, pero nada más. Esto maximizará el valor creado para la inversión realizada hasta la fecha. Daremos pequeños pasos sencillos hacia nuestro objetivo y mitigaremos los fracasos a medida que ocurran. Crearemos algo de lo que estemos orgullosos y lo mantendremos a largo plazo a un costo razonable.

Comunicación: Todos formamos parte del equipo y nos comunicamos cara a cara a diario. Trabajaremos juntos en todo, desde los requisitos hasta el código. Crearemos juntos la mejor solución posible a nuestro problema.

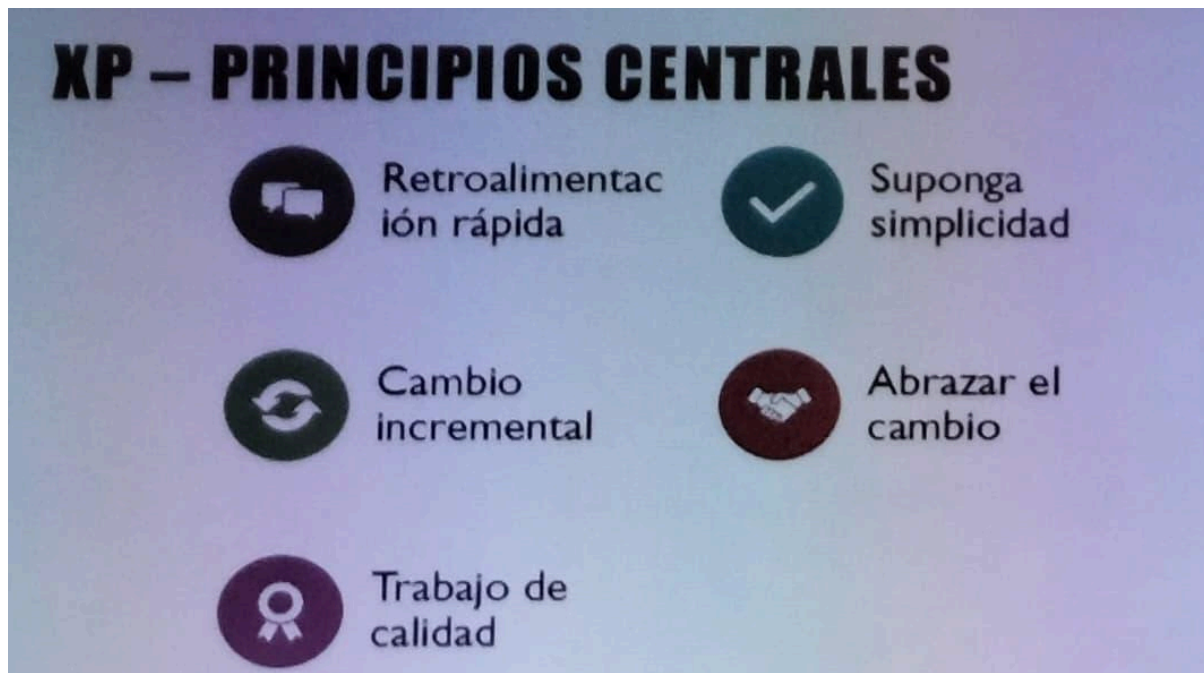
Retroalimentación: Nos tomamos en serio cada compromiso de iteración, entregando software funcional. Demostramos nuestro software de forma temprana y frecuente, escuchamos atentamente y realizamos los cambios necesarios. Hablamos del proyecto y adaptamos nuestro proceso a él, no al revés.

Respeto: Todos dan y reciben el respeto que merecen como miembros valiosos del equipo. Todos aportan valor, incluso con simple entusiasmo. Los desarrolladores respetan la experiencia de los clientes y viceversa. La gerencia respeta nuestro derecho a asumir responsabilidades y tener autoridad sobre nuestro propio trabajo.

Valentía: Diremos la verdad sobre el progreso y las estimaciones. No documentamos excusas por el fracaso porque planeamos tener éxito. No tememos a nada porque nadie trabaja solo. Nos adaptaremos a los cambios cuando se presenten.

¿Qué lecciones hemos aprendido sobre la implementación de XP hasta ahora? 🤖👤

PRINCIPIOS CENTRALES

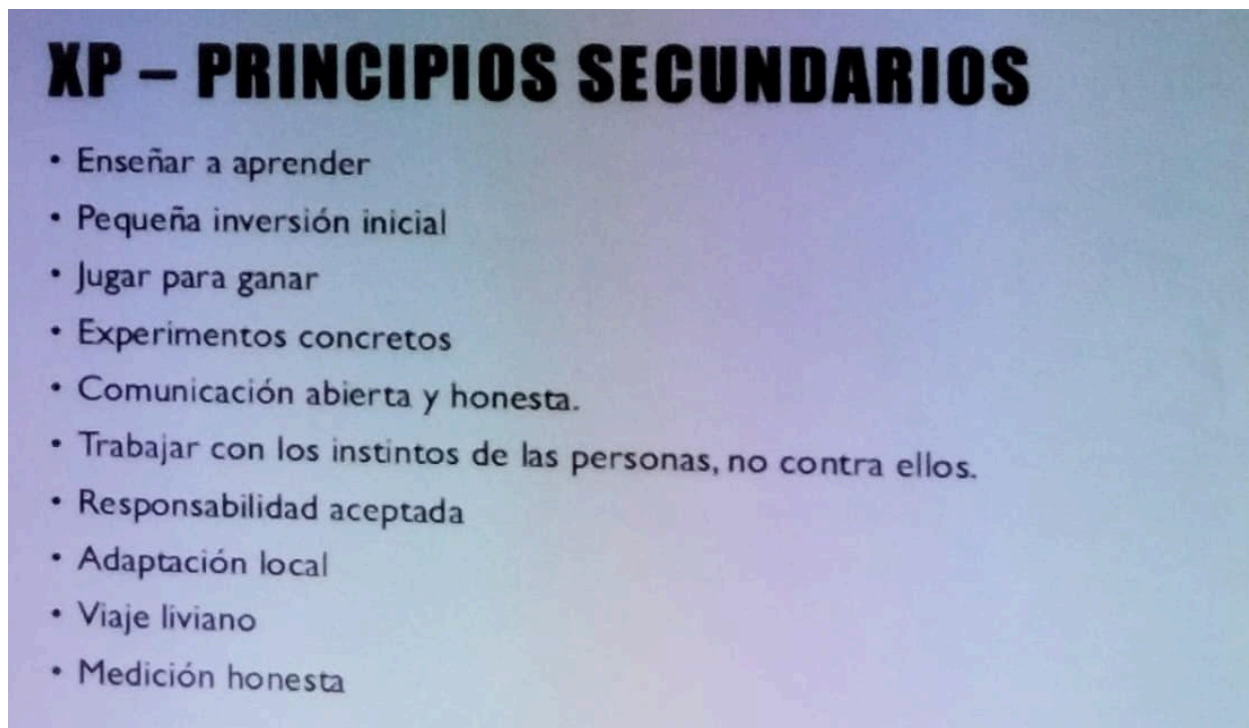


- Cambio incremental -> 2 semanas como tiempo estimado de entregas. Máximo 4 semanas.
- Trabajo de calidad -> permite mantener al equipo motivado

CARST (mnemotecnía)

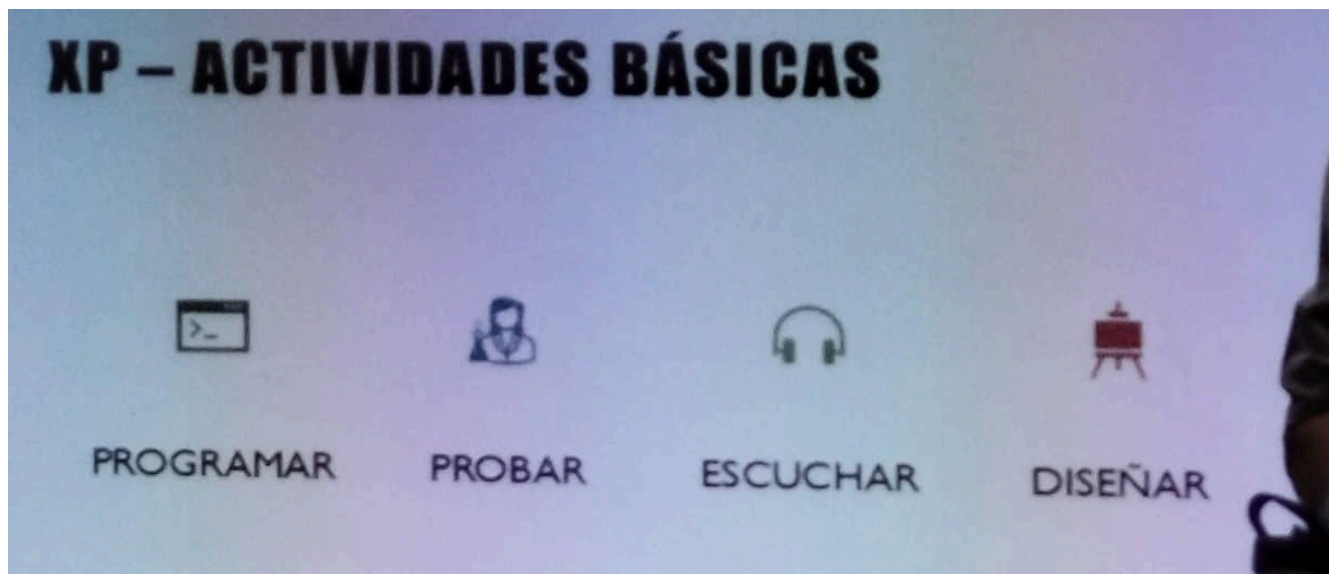
CiAcRrSTc

PRINCIPIOS SECUNDARIOS



- **Pequeña inversión inicial** -> Los proyectos que tienen mucho presupuesto inicial, se desenfocan. Es mejor arrancar con un presupuesto acotado (al menos al principio del proyecto) para poder enfocarme
- **Adaptación local** -> cambia para cada proyecto
- **Viaje liviano** -> solamente mantener los artefactos mínimos e indispensables. Si voy a hacer diagramas de clases, que sea porque lo necesito. El único artefacto necesario es el **código**, para muchos proyectos esta es la documentación. Todos los demás artefactos son extra y se debe evaluar si se necesita

ACTIVIDADES BÁSICAS



- El libro no habla de ello en detalle, pero podemos inferirlo
- Programar -> actividad principal
- Probar -> es central, cómo voy a saber si mi código funciona? Esto es sobresaliente para el framework dado que aunque no sea rigurosa, la etapa de pruebas sigue siendo central
- Escuchar -> feedback, retroalimentación, al cliente, etc.
- Diseño -> se hace muy poco foco. Se hace solo para mejorar lo que ya tengo hecho

PPED (mnemotecnica)

PRÁCTICAS CENTRALES(12)

XP – PRÁCTICAS CENTRALES

- El juego de la planificación
- Entregas pequeñas
- **Metáfora**
- **Diseño simple**
- Pruebas
- **Refactorización**

XP – PRÁCTICAS CENTRALES

- Programación en pares
- Propiedad colectiva
- Integración continua
- Semana de 40 horas
- Cliente en el sitio
- **Estándares de codificación**

(En negrita están destacadas las que son centrales para la materia) (4)

- **Metáfora** -> es una herramienta que usa Xp para tratar de explicar el sist al equipo
- **Diseño simple** -> buscar siempre solución más simple
- Pruebas -> central para XP
- **Refactorización** -> organizar y reutilizar código
- Programación de a pares -> característico de XP
- **Estándares de codificación** -> (para nosotros está vinculado a la materia)

XP – METÁFORA

- Historia compartida de cómo debería funcionar el sistema
- La metáfora reemplaza el concepto de arquitectura
- El sistema se define mediante una o varias metáforas



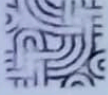
Manufacturing Metaphor

LinuxStationsBinaParts is one of approximately eight ProvenSystemMetaphors for UserInterface design.

An AnalogyBetweenProgrammingAndManufacturing is broader. It compares

- the entire process of designing / compiling / testing / selling a program, with
- the entire process of designing / making / inspecting / selling a physical product.

CategoryMetaphor



Shopping Cart Metaphor

Your key objects are "customer" (a party who buys from you) "items" (individual things to buy), "shopping cart" (a collection of items associated with a specific customer).

What other objects -- "checkout manager"?

A "credit card" object comes to mind ;)

Aha! Payment! What object is responsible for accepting payment?

See ProvenSystemMetaphors, GenericBusinessFrameworkUnobtainable

- El líder el proyecto lo utiliza para explicar cómo funciona el sistema, utilizando una metáfora sin utilizar términos técnicos y explicando a grandes rasgos la arquitectura
 - Da un ejemplo de ecommerce vs mercado físico al que se va a comprar.
- No se usa demasiado dado que para sistemas muy grandes se vuelve complicado
- De las prácticas que vemos es la menos utilizada

Metáforas

Los equipos XP desarrollan una visión común de cómo trabaja el sistema, que se denomina **metáfora**. En el mejor de los casos, la metáfora es una descripción simple de cómo el programa trabaja el programa, por ejemplo "este programa trabaja como una colmena de abejas, que salen afuera por polen y lo traen de regreso al panal" como una descripción para un sistema de recolección de información basada en agentes.

A veces no aparece una metáfora tan imaginativa. En cualquier caso los equipos XP utilizan sistemas de nombres comunes para asegurarse de que todos comprenden como funciona el sistema, y donde encontrar la funcionalidad que uno busca, o donde ubicar el lugar apropiado para agregar nueva funcionalidad.

XP – DISEÑO SIMPLE

- El diseño correcto es aquel que
 - Ejecuta todas las pruebas.
 - No tiene lógica duplicada.
 - Declara todas las intenciones importantes de los programadores.
 - Tiene la menor cantidad de clases y métodos posibles.
- Consejo opuesto a: "Implemente para hoy, diseñe para mañana".
 - No especular

- Cuando un diseño es simple? cuando ejecuta todas las pruebas -> acá hacemos énfasis en **pruebas**
- Declara todas las intenciones importantes de los programadores -> si el diseño está bien hecho, me permite comunicar cuestiones como arq. del sistema
- Consejo opuesto a "Implemente para hoy, diseñe para mañana" -> lo opuesto a esto. Diseñar para ahora, no ir por problemas que no tenemos hoy

Diseño Simple

Los equipos XP desarrollan software de diseño simple. Un equipo XP mantiene el diseño para soportar específicamente la funcionalidad requerida actualmente para el sistema.

El diseño en XP no es una actividad que se realice una única vez al comienzo del desarrollo. Hay pasos de diseño en cada Planificación de Entregas y Planificación de cada Iteración. El diseño es un proceso incremental e iterativo y se lleva a cabo por revisiones periódicas y refactorización.

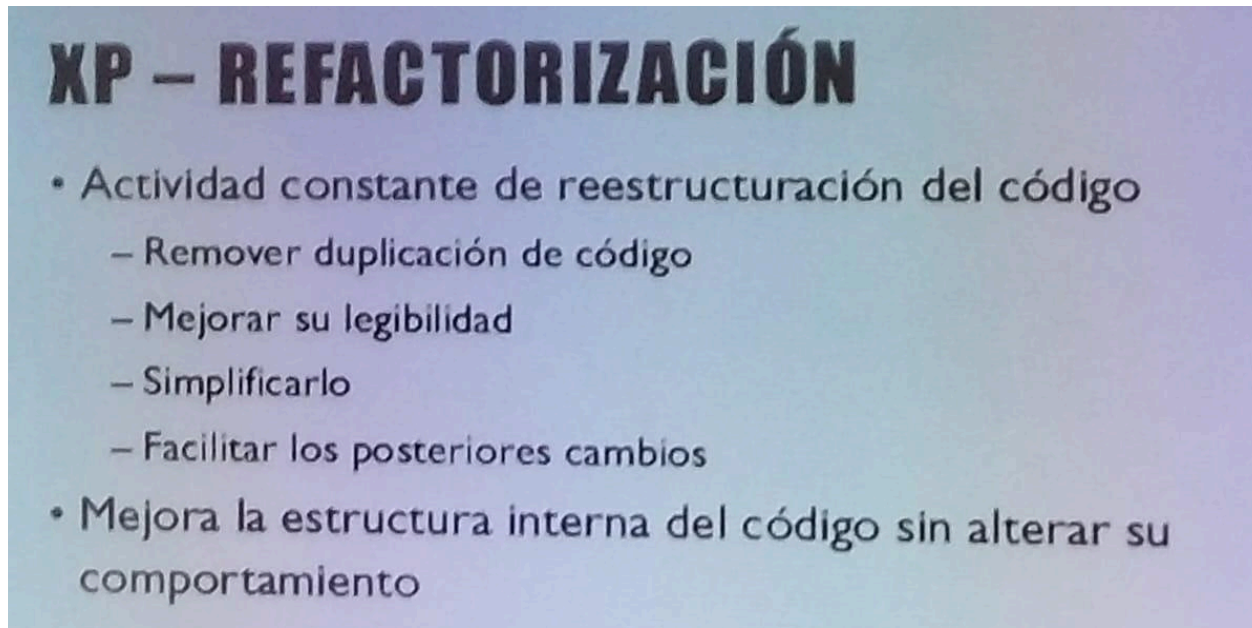
Diagramas (nose de donde saco este)

XP – DIAGRAMAS

- La estrategia de XP es que cualquiera puede diseñar con imágenes todo lo que quiera,
- pero tan pronto como surge una pregunta que puede responderse con código, los diseñadores deben recurrir al código para obtener la respuesta.
- Las imágenes no se guardan.
- Los diagramas esconden el código, no se sabe si corren los casos prueba

- Está bien hacer diagramas, pero el código debe ser la propia documentación del sistema. Debería estar tan legible y claro que me debería servir para comunicar lo que hace
- Los diagramas sirven para comunicar de manera eficiente a personas que no son desarrolladores
- Pueden ser descartables, mientras tenga el código

Refactorización



XP – REFACTORIZACIÓN

- Actividad constante de reestructuración del código
 - Remover duplicación de código
 - Mejorar su legibilidad
 - Simplificarlo
 - Facilitar los posteriores cambios
- Mejora la estructura interna del código sin alterar su comportamiento

- De los métodos más vinculados con la materia

Pasamos por arriba estas secciones

<https://refactoring.guru/es/refactoring/smells>



```

graph TD
    Start([Start]) --> PairUp[Pair Up]
    PairUp --> CreateTest[Create a Unit Test]
    CreateTest -- "Failed Unit Test" --> PairProg[Pair Programming]
    PairProg -- "Passed Unit Test" --> CreateTest
    PairProg -- "Simple Code" --> Refactor[Refactor Mercilessly]
    Refactor -- "Complex Code" --> PairProg
    PairProg -- "Changes Part" --> MovePeople[Move People Around]
    MovePeople -- "We Need Help" --> PairProg
    MovePeople -- "Simple Design" --> CreateTest
    CreateTest -- "Complex Problem" --> MovePeople
    PairProg -- "New Unit Tests" --> CI[Continuous Integration]
    CI -- "New Functionality" --> PairProg
    PairProg -- "Run All Unit Tests" --> Acceptance[Acceptance Test]
    Acceptance -- "Run Failed Acceptance Test" --> PairProg
    Acceptance --> End([100% Unit Tests Passed])
  
```

The diagram illustrates the TDD cycle and its integration with other development practices. It starts with 'Pair Up' leading to 'Create a Unit Test'. From 'Create a Unit Test', a 'Failed Unit Test' leads back to 'Pair Programming', while a 'Passed Unit Test' leads forward. 'Pair Programming' involves 'Simple Code' and 'Complex Code', leading to 'Refactor Mercilessly'. 'Pair Programming' also involves 'Changes Part' and 'We Need Help', leading to 'Move People Around'. 'Move People Around' leads back to 'Create a Unit Test' via 'Simple Design' and 'Complex Problem'. 'Pair Programming' leads to 'Continuous Integration' via 'New Unit Tests' and 'New Functionality'. 'Pair Programming' also leads to 'Acceptance Test' via 'Run All Unit Tests' and 'Run Failed Acceptance Test'. 'Acceptance Test' leads to '100% Unit Tests Passed'.

Una oruga está perfectamente diseñada para comer grandes cantidades de follaje, pero no puede encontrar pareja; debe transformarse en mariposa antes de poder buscar a otras de su especie en el cielo. Abandona tus ideas preconcebidas sobre cómo debería o no debería ser el sistema e intenta visualizar el nuevo diseño a medida que emerge ante ti. 

Hay algo de filosofía zen en la refactorización. Al principio es difícil porque debes ser capaz de desprenderte de ese diseño perfecto que tenías.

XP – ESTÁNDARES DE CODIFICACIÓN

- XP enfatiza la comunicación de los programadores a través del código,
- Indispensable que se sigan ciertos estándares de programación
 - del equipo,
 - de la organización
 - estándares reconocidos para los lenguajes de programación
- Mantienen el código legible para los miembros del equipo facilitando los cambios.

-(Lo dejo para que lo veamos y ampliemos)

Codificación Estándar

Los equipos XP siguen una codificación estándar común, de tal modo que todo el código en el sistema parece ser escrito por la misma persona. Además la codificación estándar favorece la propiedad colectiva del código.

Propiedad Colectiva del Código

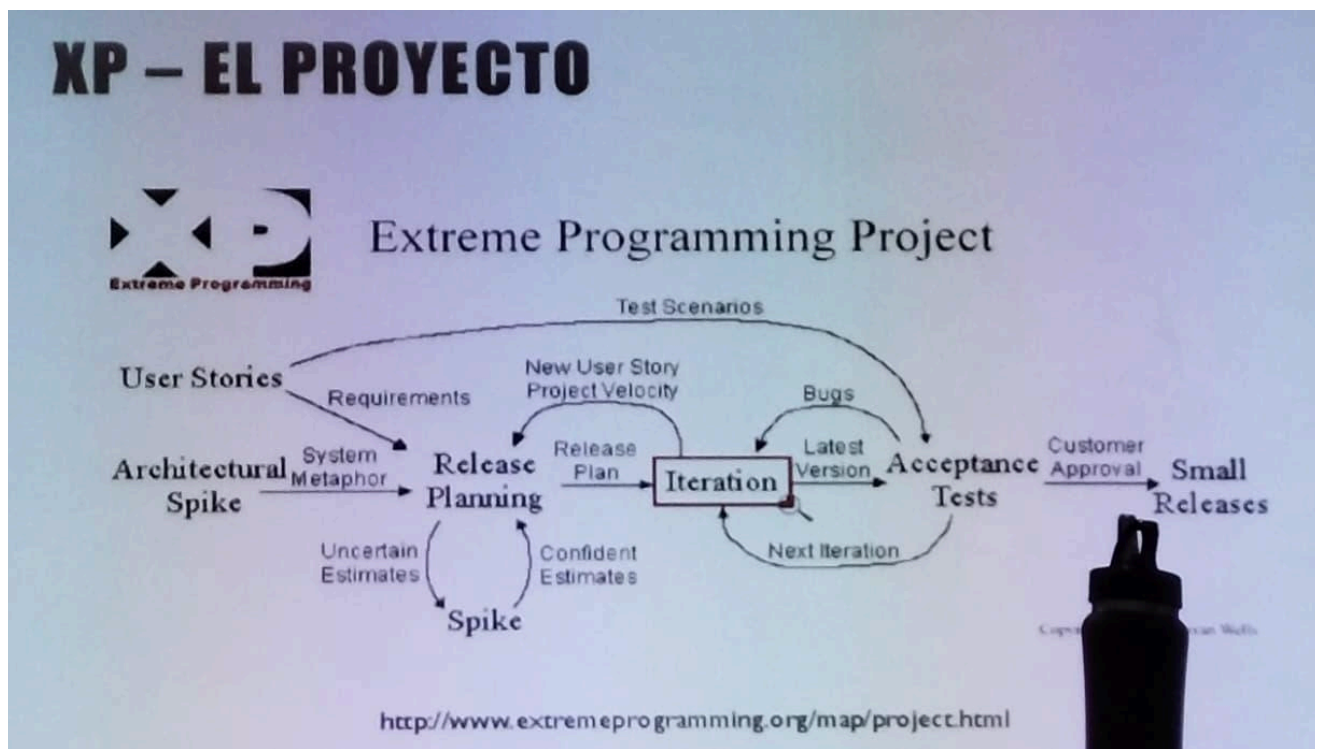
Propiedad Colectiva del Código

En un proyecto XP, cualquier par de programadores puede mejorar cualquier código en cualquier momento. Esto significa que todo el código tiene el beneficio de la atención de mucha gente, lo cual incrementa su calidad y reduce sus defectos.

La propiedad colectiva del código puede ser un problema si la gente trabaja a ciegas en código que no comprende. XP evita este problema a través de dos técnicas: las *pruebas del programador* para descubrir errores, y la *programación de a pares*, en este caso la mejor manera de trabajar con código no familiar es trabajar de a par con el experto.

- Es más fiable que asignar a una sola persona la supervisión de clases específicas, sobre todo porque una persona puede abandonar el proyecto en cualquier momento.
- Nadie se convierte en un cuello de botella para los cambios.
- Ningún sistema complejo puede estar en la mente de una sola persona.
- Gracias a las pruebas automatizadas, ahora puede confiar en el conjunto de pruebas para supervisar todo su repositorio de código. Antes de que se publique cualquier código, este debe superar todas las pruebas al 100%.

El proyecto

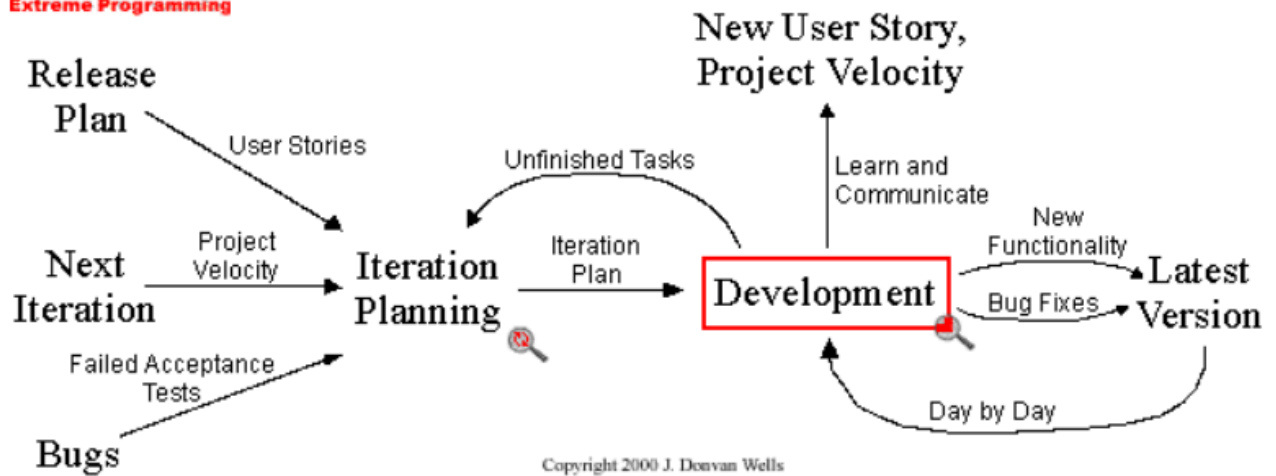


- Deja una URL para que podemos ver en detalle cómo funciona un proyecto (le vas dando click a las palabras y te abre nuevas secciones)
- <http://www.extremeprogramming.org/map/project.html>



Iteration

Zoom Out



Diseño

XP – DISEÑO

- **Simplicidad**
- Elija una **metáfora del sistema**
- Utilice **tarjetas CRC** para sesiones de diseño
- Cree **soluciones de picos (spike)** para reducir el riesgo
- No se agrega ninguna funcionalidad temprano
- **Refactorizar** siempre que sea posible

<http://www.extremeprogramming.org/rules.html>

<http://www.extremeprogramming.org/rules.html>

- spikes -> Despejar incertidumbre de algo que desconozco. Hacer una pequeña prueba técnica para ver la viabilidad. Suele ser una prueba técnica
 - Ejemplo, tengo que hacer un sistema de facturación pero nunca integre AFIP
 - > pruebo algo puntual de la API

Spike

Crear una solución “spike”

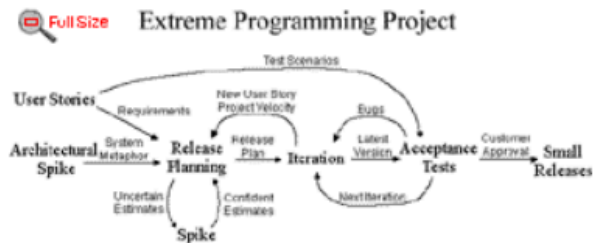
Crear soluciones “spike” para representar cuestiones técnicas o de diseño. Una solución “spike” es una solución muy sencilla para experimentar soluciones potenciales. Su objetivo es reducir el riesgo de problemas técnicos o incrementar la confiabilidad de una estimación sobre una historia de usuario.



Crear una solución Spike

Crea prototipos para encontrar soluciones a problemas técnicos o de diseño complejos. Un prototipo es un programa muy sencillo que explora posibles soluciones. Céntrate en el problema en cuestión e ignora todo lo demás. La mayoría de los prototipos no son lo suficientemente buenos como para conservarlos, así que probablemente tendrás que descartarlos. El objetivo es reducir el riesgo de un problema técnico o aumentar la fiabilidad de la estimación de una historia de usuario.

Cuando una dificultad técnica amenace con retrasar el desarrollo del sistema, asigna a dos desarrolladores al problema durante una o dos semanas para minimizar el riesgo potencial. 🧑🏻‍💻



Limitaciones

XP – LIMITACIONES

- La **falta de diseño formal** puede ser peligrosa para grandes sistemas
- Dificultad en proyectos con **gran cantidad de usuarios**
- Está dirigido explícitamente al **desarrollo de software únicamente**. Sistemas que requieran hardware específico, inevitablemente demandan especialistas.
- Demanda **personas sobresalientes**, de múltiples capacidades.
- Pérdida de documentación. Limitada al código.
- Falta un proceso de revisión estructurada.
- Falta de plan de calidad.
- No describe profundamente sus métodos.

- No tiene etapa de diseño formal, para grandes proyectos puede ser problemático
- XP a diferencia de SCRUM se concentra en el desarrollo del SW. SCRUM se mete en la gestión del proyecto y no en el código, XP no se mete en la gestión y si en el código
- Pérdida de documentación del diseño: limita XP a pequeños programas y **dificulta las oportunidades de reutilización.**

. Falta un proceso de revisión estructurada

- **Qué significa:** En los métodos tradicionales (cascada), existen etapas formales donde el proyecto se "detiene" para ser revisado y aprobado por un comité o gerente antes de avanzar (ej. "Firma de aprobación de requisitos").
- **En XP:** La revisión es continua e informal (mediante *Pair Programming* o *Feedback* constante). No hay un momento específico donde alguien externo ponga un sello de "Aprobado" sobre un documento.
- **Por qué es una limitación:** Para empresas muy burocráticas o auditorías estrictas, esta falta de "hitos de control" formales puede generar desconfianza o desorden. 

2. Falta de plan de calidad

- **Qué significa:** Un "Plan de Calidad" (QA Plan) suele ser un documento extenso que define métricas, estándares y procedimientos de auditoría independientes al desarrollo.
- **En XP:** La calidad es responsabilidad de los programadores a través del código (TDD, Tests unitarios). No existe un departamento de QA separado que planifique cómo "controlar" a los programadores.
- **Por qué es una limitación:** Si el cliente exige un documento formal que garantice cómo se asegurará la calidad antes de empezar, XP no lo tiene. Confía en la práctica técnica diaria más que en el plan documental. 

3. No describe profundamente sus métodos

- **Qué significa:** XP es una "metodología liviana". Te da los valores (ej. Simplicidad) y las prácticas (ej. Programar de a pares), pero no es un manual de instrucciones paso a paso para cada situación posible.
- **En XP:** Se asume que el equipo es experto y sabrá cómo resolver los detalles del día a día.
- **Por qué es una limitación:** A diferencia de metodologías pesadas (como RUP o PMBOK) que te dicen exactamente qué hacer en cada escenario, XP deja muchos huecos que deben ser llenados por la experiencia y el instinto del equipo. Si el equipo es junior, pueden perderse fácilmente. 

Gaona dice que nos enfoquemos en las prácticas centrales

- Metáfora
- Diseño simple
- Refactorización
- Estándares de codificación -> ver que son los spikes